

School of Computer Science and Engineering, UNSW



Never Stand Still

WK03-01: Smart Contracts and Oracles

Software Architecture for Blockchain Applications,
COMP6452

Salil Kanhere, Nadeem Ahmed

Agenda

- Smart Contracts
- Smart contracts and legal contracts
- Tokens
- Oracles

- We start with a discussion on smart contracts and their relationship (or lack of relationship) with legal contracts.
- Next, we'll have a very high-level discussion about tokens.
- Finally, we'll discuss oracles that enable a blockchain/smart contract to interact with off-chain components of a software system.

Smart Contracts (SCs)

Blockchain can be a computational platform

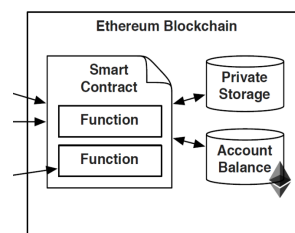
- More than a distributed database

User-defined code, deployed on & executed by whole network

- Can maintain its state in the ledger
- Can hold & transfer digital assets, managed by the contract itself

Analogy – Java program

- SC code – a class
- Deployed contract – an object
- Many software design patterns still apply



- Among the 4 functions of blockchains discussed in 1st class, Smart Contracts (SCs) use the blockchain as a computational platform.
- A smart contract is essentially a small, user-defined code/program/script deploy and execute on blockchain nodes.
 - smart contract has its state space in the ledger to maintain the data. Given the storage and computational ability, a smart contract can hold and transfer digital assets/states, managed by the contract itself.
 - The program logic can enact decisions on complex business conditions. E.g., automatic partial payment can be triggered as a product moves along a supply chain.
- smart contracts reflect the object-oriented paradigm like in Java, i.e., smart contract code is like a class and deployed contract is an object. So, when we say smart contract, we typically refer to the deployed contract.
 - Being a program, many software design patterns still apply to smart contracts, e.g., a factory contract spawning new smart contracts.

Smart Contracts

Code is deterministic

- Same ledger state & inputs result in same state changes & outputs
- However, state may not be deterministic from viewpoint of TX caller

Execution is only triggered via a TX

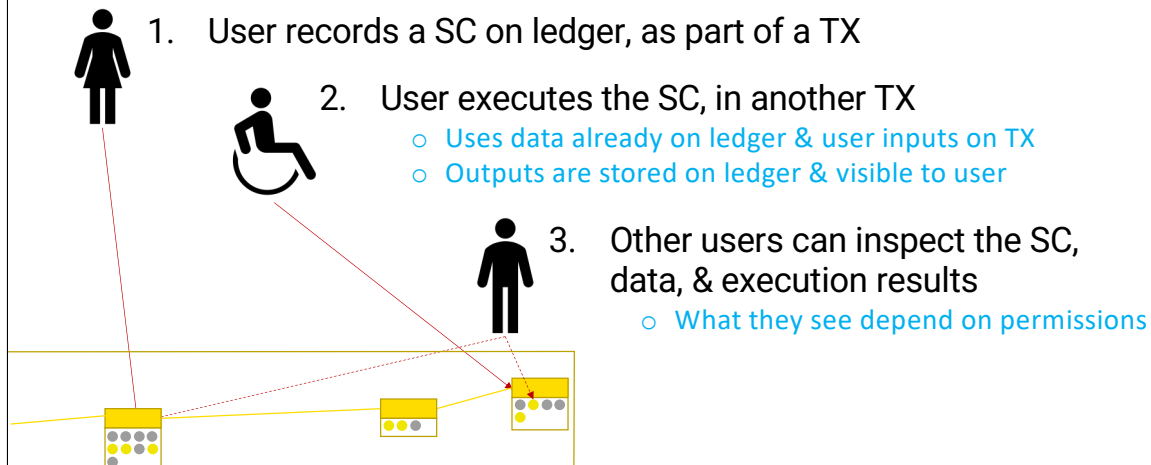
- Aka a message call
- A smart contract can invoke other smart contracts

Why can SC execution be trustworthy?

- A contract is deployed as data in a TX → Immutable, transparent, & deterministic code
- All inputs are through TXs & current ledger state → Transparency & integrity

- The smart contract code is deterministic, where the same inputs (inputs in transaction and ledger state) result in the same ledger state changes and outputs.
 - However, it may not be deterministic from the caller's viewpoint. E.g., other transactions may get executed before a given caller's transaction changes the input state.
- Execution of a smart contract can be triggered only via a transaction.
 - In Ethereum, a transaction used to trigger/call a function on a smart contract is called a *message call*.
 - a smart contract can also invoke other smart contracts.
- We can consider smart contract execution to be trustworthy due to the following reasons:
 - A contract is deployed as data in a transaction; hence, the same immutability property applies to the smart contract code. Also, as transaction and the ledger state are transparent, the code is transparent. Further, the code is deterministic; hence, we can precisely know its output and resulting ledger change.
 - All inputs are only through transactions and the current state. Hence, the inputs of the contract are transparent.
 - In Ethereum, the results of transactions (including function calls) are captured in the State and Receipt Merkle trees, which are part of the consensus.
 - A contract's state can only be changed via a transaction that invokes a function on the contract. Hence, if access control is properly set, we can guarantee that no one can change the state without going through the contract and applying relevant business logic. This means integrity is preserved.
 - This is different from a database, where one can bypass the application and go and update the database. You can attempt to do the same in a blockchain node, but it won't be synched with other nodes without going through the consensus process which requires a transaction.

Life of a Program on the Ledger



WK 03-01

5



- A user records a smart contract on the ledger as part of a transaction.
- Then the same user or a different one (depending on the access control settings on the smart contract) can execute the smart contract in another transaction.
 - When we say execute a smart contract, we mean to execute a specific function on the smart contract, e.g., *mint* or *transfer* tokens.
 - A common misconception is that smart contracts are “self-executing” or “autonomous”. That is not true of most platforms. Instead, smart contracts usually have to be invoked by someone.
 - But anyone can attempt to invoke any smart contract, and whether it results in the ledger state update depends on the access control settings on the smart contract. E.g., while anyone can call the *mint* function on a token contract, only the user with the *minter* role can mint tokens. Others will experience an error.
- When executing a smart contract function, we depend on 2 forms of data:
 - One is the data already on the ledger.
 - Other is data sent by the user as part of the transaction.
- Other users can inspect the smart contract, data, and results of execution from the 1st transaction onwards. However, on permissioned blockchains, there are restrictions on what others can see, e.g., a 3rd party may see a transaction happening, but they may have access to the transaction input or ledger data.

What are SCs Good for?

Recall, blockchains are good for ...

1. New (better?) ways to work together
2. Exclusive control of digital assets

Smart contracts same

- Deterministic, immutable, & transparent code & data

Not just shared data. Also, shared behaviour, i.e.,

1. New (better?) programmable ways to work together
2. Programmable exclusive controls, for programmable digital assets
 - Smart contracts can control digital assets & can also be digital assets

- So, what are smart contracts good for? If we recall, what blockchains are good for, we said they are good for
 - New or better ways to work together (better depends on the user's viewpoint)
 - Exclusive control of purely digital assets like cryptocurrencies and NFTs.
- Smart contracts are the same.
 - Like data on a blockchain, smart contract behaviour is deterministic and immutable. Also, all inputs and ledger state changes are transparent. Hence, they are also trustworthy like data on a blockchain (as far as input data are valid)
- However, smart contracts are more than data, they also give shared behaviour, as others can execute the contract and can see the smart contract code, data, and execution results, i.e.,
 - New or better programmable ways to work together. Define multiparty business logic on how the data should be managed.
 - Programmable exclusive controls, for programmable digital assets.
 - smart contracts can control digital assets (e.g., tokens) and can also be digital assets. E.g., Forsage is a network marketing scheme (essentially a pyramid scheme) in that participants buy ownership of the contract to earn from future participants joining the contract.

Smart Contract Interactions

Creation/deployment TX

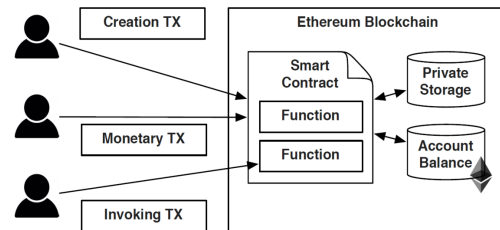
- Contains SC code & “To” address is set to Null
- Once included, identified by account address
- SC account contains
 - Executable code
 - Internal state
 - Cryptocurrency balance

Monetary TX

- Can send cryptocurrency into SC

Invoking TX contains

- Name of function being invoked & its parameters as TX data payload
- Invoked function may call 1 or more functions in other SCs
- Assets held by account address are controlled by executable code



- As shown in the figure, there are 3 types of smart contract-related transactions. We discuss these in the context of Ethereum.
 - All transactions must originate from an external/user account.
 - All these transactions need to be signed by authorised users and need to pay transaction fees (if transaction fees apply).
- First, the smart contract needs to be created or deployed onto the blockchain using a *contract creation transaction*.
 - The data file of the transaction contains the smart contract code.
 - The “To” address in the transaction is set to Null.
 - Once a smart contract is included in a block, an account address is assigned as per protocol rules. This address is called the smart contract address. In most blockchains, smart contract addresses are not different from user addresses and just by looking at an address you can’t differentiate between a user and a smart contract address.
 - However, smart contract addresses don’t have a corresponding private key to issue transactions by anyone.

- The smart contract account contains the executable smart contract code, storage to store internal state, and cryptocurrency balance (e.g., amount of Ether, only if the blockchain has a native cryptocurrency).
- A *monetary transaction* can be used to transfer cryptocurrency such as Ether to a smart contract.
 - Once received, a smart contract can hold and transfer crypto assets. However, no user can transfer that cryptocurrency from the contract's address, as there is no private key associated with that address.
 - blockchains that do not have an underlying cryptocurrency (Hyperledger Fabric) do not support Monetary transactions.
- Finally, an *invoking transaction* is used to invoke a function on smart contract code.
 - smart contracts need to be externally invoked using invoking transactions. They can't execute themselves. While we typically say that a smart contract can automatically execute/enforce some business logic, in practice, it must be triggered by a user-generated transaction. Of course, you can automate an off-chain (i.e., external) system to automatically issue an invoking transaction when the conditions are right, e.g., when a timer expires.
 - Such transactions need to send to the contract address while defining the function to call and with what parameters.
 - The invoked function may call 1 or more functions in other smart contracts.
 - The involved smart contract function can then manipulate the assets held by the smart contract's address, e.g., transfer Ether to a user.

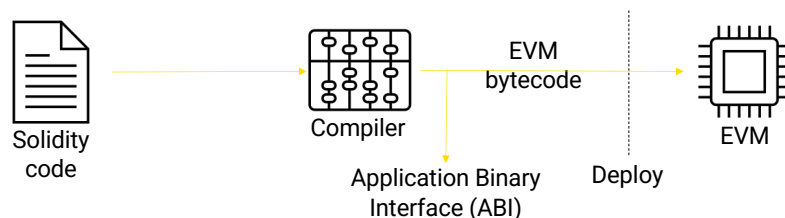
Smart Contract Development in Ethereum

SCs in Ethereum are typically developed in a high-level language

- They are compiled to bytecode before deployment & execution
- Runs on Ethereum Virtual Machine (EVM) on each node

Solidity is the most popular SC language for Ethereum

- High-level, Turing-complete, object-oriented language; syntax is like JavaScript
- Statically typed, supports inheritance, libraries, & complex user-defined types



WK 03-01

8



- smart contracts in Ethereum are typically developed using a high-level language such as Solidity and Viper.
- Code is then compiled to bytecode (i.e., low-level code understood by a virtual machine) before deployment and execution.
 - You could certainly develop smart contracts in the supported low-level language too. But it's a tedious process
- The bytecode is then executed on an execution environment called Ethereum Virtual Machine (EVM) on each node. You may consider this similar to running Java bytecode on the JVM.
- Solidity is the most popular smart contract language for Ethereum.
 - Today, several other blockchain platforms such as Hyperledger Besu, Binance Smart Chain, and Concord also support Solidity.
 - Solidity is a high-level, Turing-complete, object-oriented language. Syntactically it's like JavaScript.
See https://en.wikipedia.org/wiki/Turing_completeness
 - Solidity is statically typed, supports inheritance, libraries, and complex user-defined types
- These are some of the resources to learn Solidity.
 - Solidity official documentation page: <https://solidity.readthedocs.io/en/latest/>
 - Official examples: <https://solidity.readthedocs.io/en/latest/solidity-by-example.htm>
 - Other tutorials, e.g.:
https://ethereumbuilders.gitbooks.io/guide/content/en/solidity_tutorials.html
 - Best practices from Consensys (a company) <https://consensys.github.io/smart-contract-best-practices/>

Transaction Format

```
pragma solidity >=0.7.0 <0.8.0; // compiler inst.: language & version

contract SimpleStorage { // contract name
    uint storedData;      // variable declaration

    /// getter, callable by anyone
    function get() public view returns (uint) {
        return storedData;
    }

    /// setter, callable by anyone
    function set(uint x) public {
        storedData = x;
    }
}
```

WK 03-01

9



- Following is a simple Solidity code.
- *pragma* is an instruction to the compiler (or what's called a pre-process directive) saying that this code should only be compiled with compiler version 0.6 to 0.8.
- *The contract* is like the Class statement. So, the name of our contract is SimpleStorage.
- Here we defined a variable of the type unsigned integer (i.e., nonnegative integer).
- Lines starting with *//* and *///* are comments
- These are getter and setter functions for the variable. They can be called by anyone on the blockchain network as they are marked as 'public' functions
- *get()* is also marked as 'view' indicating that it can't modify the state, i.e., real-only function.
- .

A “Real” Smart Contract

! Do not use in production !

```
contract Escrow {
    address payable public payer;
    address payable public recipient;
    address public approver;

    constructor (address _approver, address payable _recipient) payable {
        payer = msg.sender;
        approver = _approver;
        recipient = _recipient;
    }

    function pay() public {
        require(msg.sender == approver);
        recipient.transfer(address(this).balance);
    }

    function amount() public view returns (uint256) {
        return address(this).balance;
    }
}
```

Create a 1 ETH escrow with
(new Escrow){value:1 ETH}
(approver, recipient)



payer



Approves
payment with
pay()

approver

this



recipient

1 ETH is
transferred
during pay()

WK 03-01

10



- Here's a simplified but more real-world version of an escrow contract written in Solidity.
 - A real-world escrow is a contractual arrangement in which a third party receives and disburses an asset for the primary transacting parties, with the disbursement dependent on conditions agreed to by the transacting parties.
- The contract manages the interaction between 3 parties, the payer, approver, and recipient.
- The payer deploys the escrow contract while transferring 1 ETH (Ether) and set the approver and recipient address.
 - The *payable* keyword says that the contract can accept ETH.
 - When the contract is deployed (i.e., created on the blockchain), the constructor is executed. Within the constructor code, we can see that the message/transaction sender is set as the payer. Approver and recipient are set as the input to the constructor.
 - Contract balance is not explicitly set as it is handled by Ethereum base protocol that manages transfer and store of ETH.
- Once the desired payment condition is satisfied, the approver calls the *pay()* function to transfer the funds (i.e., 1 ETH) to the recipient.
 - This function 1st check whether the *pay()* function is called by the designated approver where the message sender's address must match to address of the approver. This is essentially access-control check to ensure only the approver calls this function.
 - Then it transfers the entire ETH balance (the contract finds its balance using *address(this).balance*) held by the contract to the recipient address.
- There's also an *amount* function to check the balance of the escrow contract. While this is not essential (as it can be checked with other means), it would help if the escrow contract is used with another smart contract. Note the 'view' keyword which marks the function as read-only.
- This contract is incomplete as it does not have a way of transferring funds back to the payer if the payment condition is not satisfied. It's omitted for simplicity. The tutorial example contains such a feature

Solidity - Features

Can specify interfaces, by having at least one un-implemented function

```
contract Base { function foo(); }    // an interface
contract Derived is Base { function foo() {} } // implements the interface
```

Can overload functions

- But might not be a good idea – hard to understand code

Arrays work like in Java/JavaScript

```
uint[3] public data;
data[0] = 0; ...
```

Constructor is executed once when a contract is created

```
constructor(uint i) public{
...
}
```

- Here are a few additional details about Solidity.
- Like many other languages, you can specify interfaces. E.g., in class Base, we specify the interface foo while it's implemented in class Derived.
- You can overload functions too but remember they can lead to confusion code.
- Arrays work similarly to Java/JavaScript.
- The constructor is executed once when a contract is created. This is useful, as you can set the owner of the contract within the constructor. You can skip the constructor if there's no need to set any state at the beginning of the contract.

Solidity – Example 2

```
pragma solidity >=0.4.22 <0.6.0;
contract Ballot {
    struct Voter {
        uint8 vote;
        bool voted;
        uint weight;
        address delegate;
    }
    struct Proposal {
        uint voteCount;
    }
    address chairperson;
    mapping(address => Voter) voters;
    Proposal[] proposals;
    /// Create a new ballot with $(_numProposals) different proposals.
    constructor(uint8 _numProposals) public {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        proposals.length = _numProposals;
    }
}
```

Source: <https://remix.ethereum.org/>

WK 03-01

12



- Here's a more detailed example related to our Project 1.
- The contract name is *Ballot*.
- Here we define 2 structures called *Voter* and *Proposal*.
 - *voted* keeps track of whether the voter already voted, and the vote result is maintained in the *vote*.
 - Typically, we consider all votes equal. However, in this code, we use a *weight* variable to give disproportionate weight to votes. E.g., we can weigh one's vote based on the number of stocks he/she owns in a company.
 - If there is a delegate who can vote on behalf of the voter, we track the Ethereum address of the voter here. Note the data type is address (which is a 160-bit string). E.g., in capital markets a fund's manager may vote on behalf of the asset owner.
- The *Proposal* structure is used to keep track of *voteCount* for each proposal we

are voting for.

- A list of vote counts is maintained in the Proposals array.
- The *chairperson* is another address and is the owner or controller of the voting process
- We keep track of the list of voters using a hash map where the key is the voter's address, and the value is the Voter structure. This is like a table with 2 columns. Column 1 is the address of the voter and column 2 is the voter structure.
- When we run the Constructor at the time of smart contract deployment, we set the chairperson to the sender of the transaction. The sender is identified from `msg.sender`.
- We also set the chairperson as a voter and set the number of proposals to the value passed while calling the constructor.

Solidity – Data and Functions Visibility

2 types of function calls

- Internal call – local call from the same contract
- External call – message/TX call by user or another contract

Different state & function visibility settings

- Public – Can be called externally & internally
 - Compiler automatically creates getter functions for public state variables
- Private – Internal only (not even by derived contracts)
- Internal – Internal & derived contracts
 - Default for state variables
- External – External only
 - If `f()` is external, calling `f()` won't work, but `this.f()` works within contract

- While all data on an Ethereum smart contract is publicly visible, for the proper functioning of our smart contracts we must specify the variable and function visibility correctly. This prevents accidental state changes and can also act as some form of access control.
- E.g., Solidity provides 2 types of function calls:
 - Internal calls are local calls from the same contract.
 - External functions are triggered by a transaction (or message) send either by a user or another smart contract, i.e., called by a source outside the contract.
- There are different state and function visibility settings:
 - Public functions can be called both by transactions/calls arising externally and internally. For state variables that are marked public, the compiler will automatically create getter functions.
 - Private functions are only visible in the contract that they are declared in. They can't be used in derived contracts.
 - Internal functions can be called only from other codes in the same contract or by derived contracts (through inheritance). State variables of a smart contract are internal by default.
 - External functions can be called only from outside (either transaction from a user or another contract). Such functions can't be called within the smart contract. If function `f` is external, calling it internally is impossible. However, while `f()` will fail, `this.f()` will work as it's like the call is coming from *this* contract.

Solidity – Visibility Example

```
contract Cont1 {
    uint private data;
    function func(uint x) private returns(uint y) { return x + 1; }
    function dataSet(uint x) public { data = x; }
    function dataGet() public returns(uint) { return data; }
    function compute(uint x, uint y) internal returns (uint) { return x+y; }
}
contract Cont2 {
    function dataRead() public {
        Cont1 z = new Cont1();
        uint local = z.func(7); // error: member "func" is not visible
        z.dataSet(3);
        local = z.dataGet();
        local = z.compute(3, 5); // error: member "compute" is not visible
    }
}
contract Cont3 is Cont1 {
    function g() public {
        Cont1 z = new Cont1();
        uint val = compute(3, 5); // access to internal member from derived to parent contract
    }
}
```

Adapted from <https://www.bitdegree.org/learn/solidity-visibility-and-getters>

WK 03-01

14



- Here are some examples of data & function visibility.
- Contract 1 (Cont1) has a private variable and 4 functions with different visibility settings.
- Contract 2 uses the class Cont1 while Contract 3 extends Cont1.
- *Cont2* try to access *func* from *Cont1*, which is marked as private. Private functions are internal only hence, this call will fail.
- Calls to *dataSet* and *dataGet* will work as it's public functions.
- *compute* is an internal function, much like private from an external contract. Hence, this call will fail.
- However, it won't fail in *Cont3* as *Cont3* derives from *Cont1*, as internal functions can be called from derived contracts.

Smart Contracts – Best Practices

Follow KISS principle: keep it simple, stupid

- Make it understandable, so other technical users can start trusting your code

Follow best practices, especially around security – there is no “safety net”

Always look at online documentation

- SC languages are under very active development & changes are frequent

Make code available to potential users (e.g., open source), or at least the interface

- Many interface standards proposed, e.g., ERC-20 token standard

Interface of a SC isn't visible from the deployed code

- Solidity compiler establishes a particular structure, e.g., ABI (Application Binary Interface) that needs to be known to the caller

WK 03-01

15



- While smart contracts are essentially another piece of code, you need to be extra careful as the code is visible to others exposing any potential vulnerabilities. Also, there are minor differences in executing a code on a typical desktop vs on EVM due to differences in implementation. Such differences need to be thoroughly understood.
- Further, there's no rollback if something goes wrong due to the immutability of a blockchain. Hence, we need to be very careful.
- Following are some of the best practices:
 - Follow the KISS principle, where KISS stands for keep it simple, stupid. This essentially means making it readable/understandable so other developers and technical users can start trusting your code.
 - Follow best practices, especially around security. Due to immutability and lack of central governance, there is no safety net if something goes wrong.
 - Always look at online documentation as both Solidity and EVM are under rapid evolution.
 - You may also want to make the code available to potential users (e.g., open-source), or at least the interface and there are several interface standards such as ERC-20 and ERC-721.
 - While the interface or functions of a smart contract isn't visible from the deployed code, and there's no well-defined structure, the Solidity compiler establishes a particular structure. In Solidity, this structure is encoded into a JSON (JavaScript Object Notation) object known as the ABI (Application Binary Interface).
 - This structure needs to be known to the caller of a function to properly format the transaction. You will see in a later lab/tutorial how to use ABI.

Decentralized Applications (DApp)

Applications where main functionality is implemented through smart contracts

Backend is executed in a decentralized environment

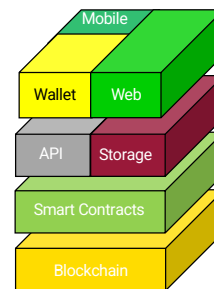
- Including decentralised storage

Frontend can be hosted as a web or mobile app

- Interacts with its backend through an API

“State of the dapps” is a DApp directory recorded on blockchain

- <https://www.stateofthedapps.com/>



- Decentralised applications or DApps are applications where main functionality is implemented through smart contracts.
- Application backend is executed in a decentralised environment like blockchain.
- As seen in the figure, even its storage may be decentralised, e.g., IPFS which stands for Inter-Planetary File System is a P2P storage solution used by some DApps.
- The frontend can be hosted as a web or mobile app. It interacts with its backend through an API.
- “State of the dapps” is a directory of DApps recorded on blockchain.
- It’s important to understand that while DApps rely on smart contracts, not all smart contracts belong to DApps.

Question

Which of the following statement(s) is True?

- ✓ A. Smart contracts are good for managing data and achieving shared behaviour among a set of parties.
- ✗ B. All smart contract languages are Turing-complete.
- ✓ C. A smart contract's ledger state may change between issuing a transaction and its inclusion in a block.
- ✗ D. In Solidity, a private function can be called from another smart contract.

WK 03-01

17



- These are the types of questions you can expect in the quiz.
- A smart contract can be used both as storage and computational elements. Hence, they are good for both managing data and achieving shared behaviour among a set of parties, e.g., an Escrow contract.
- Not all smart contract languages are Turing complete, while Solidity is Turing complete. Daml is not.
- There is some time delay between issuing a transaction and it gets included in a block (this can vary from a few seconds to many minutes and even hours). Hence, there is a possibility that another transaction may execute in the meantime changing the ledger state.
- As in typical programming, a private function cannot be called from outside the contract. Not even by a derived contract.

Agenda

- Smart Contracts
- Smart contracts and legal contracts
- Tokens
- Oracles

- Let's discuss how smart contracts may or may not relate to legal contracts.

Is a Vending Machine a “Contract”?

1. Yes
2. No, the contract is written on the front of the machine
3. No, but using the machine establishes a contract
4. No, a different legal concept applies
5. No, for some other reason
6. I don't know, I'm not a lawyer!
7. I don't know, and I am a lawyer!



- How many of you think a vending machine is a contract?
- There are different views on this, e.g., If so, where is the contract written?
- There are contracts at least between the vending machine company and the customer who wishes to install them.
- The last one is interesting, as even lawyers don't know when it comes to digital assets. That is why Treasury is undertaking a token mapping exercise to give guidance on different types of crypto assets.

Nick Szabo's Original Concept

"...contractual clauses ... can be embedded in hardware and software A canonical real-life example, which we might consider to be the primitive ancestor of smart contracts, is the humble vending machine."

"Smart contracts go beyond the vending machine in proposing to embed contracts in all sorts of property that is valuable and controlled by digital means."

"The Idea of Smart Contracts" Nick Szabo (1997)
<https://archive.vn/wIUOA>



- The idea of smart contracts existed before blockchains were invented. Bitcoin emerged in 2009, but Nick Szabo talked about smart contracts in 1997.
- He talked about the possibility of embedding contractual clauses in hardware and software.
- He then gives other examples of properties/assets that could be controlled by digital means. For example, he talked about a car security system that might let a creditor access the car, not just the owner. He also talked about conditions under which the creditor gets access, e.g., one payment missed and while the car is not being driven.
- Full article is at <https://nakamotoinstitute.org/formalizing-securing-relationships/> and extract available at <https://archive.ph/wIUOA>

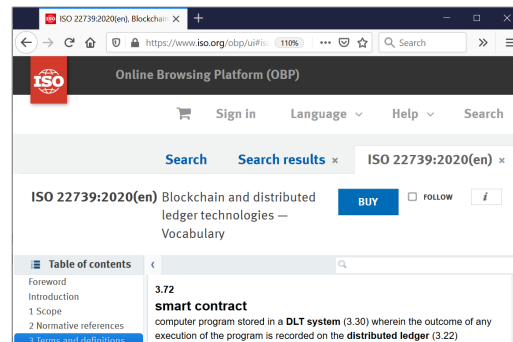
SCs in Blockchain Context

Smart contract

“computer program stored in a DLT system wherein the outcome of any execution of the program is recorded on the distributed ledger” – ISO 22739:2020

Note:

“A smart contract can represent terms in a contract in law and create a legally enforceable obligation under the legislation of an applicable jurisdiction.”



- ISO 22739:2020 standard defines a smart contract as a computer program stored in a DLT wherein the outcome of any execution of the program is recorded on the distributed ledger.
 - See ISO 22739:2020 Blockchain and Distributed Ledger Technologies – Vocabulary <https://www.iso.org/obp/ui#iso:std:iso:22739:ed-1:v1:en:term:3.72>
- The term “execution” here means the computer science idea of running a program.
- Although smart contracts are programs, they might still also be legal contracts! There is a note for this definition from ISO that says that some smart contracts might be related to legal contracts.

“Code is Law”?

Lawrence Lessig’s original idea

- “Code requires choices about values, has social implications, and should be governed like the law”
- We need societal governance for values & rules for our software, but not by government

vs. crypto-anarchist slogan

- Some people think this means “The power of code can take over from the law”
- Physical reality of running code supersedes the law

What should a smart contract do?

- What we need?
- What we thought we agreed on?
- What we said?
- What the code will force the machine to do?

Code can’t take over from law, but it should be governed by the law

- In the context of blockchains, we implement business and governance functionality in the form of smart contracts. So, the question is can we believe in code?
- The idea of “Code is Law” comes from a quote from Lawrence Lessig in Harvard Magazine (see <https://harvardmagazine.com/2000/01/code-is-law-html>). He has a book entitled “Code And Other Laws of Cyberspace”.
- He said, “Code requires choices about values, has social implications, and should be governed like the law”.
- We need societal governance for values and rules for our software, but not from the government. Even more than blockchain, we are now realising the wide-spread need for such governance with all the chaos created by ChatGPT (or generative models in general).
- Some people (aka crypto-anarchist) think this means “The power of code can take over from the law”.
- This is sort of the case because the physical reality of running code appears to supersede/replace the law. E.g., to run an escrow, you no longer need to have a contractual agreement with a 3rd party
- Code can’t take over from law, but it should be governed by the law.
- We expect a smart contract to do things like:
 - What do we need?
 - What we thought we agreed on?
 - What we said?
 - What the code will force the machine to do?
- The last one is interesting as the code will force the machine to do things that we did not anticipate. It’s not that the code is misbehaving, it’s just code can be executed in ways we did not anticipate. e.g., by giving inputs, we do not handle or execute functions in a particular order that we didn’t anticipate.

Lessons from “The DAO”

DAO: Decentralized Autonomous Organisation

A venture-funding entity itself run as a smart contract

Raised > \$150M in a month

But a “bug” (?) in the code led to a slow leak of > \$50M

Led to developers “forking” Ethereum blockchain

- Ethereum – Lessig’s view
- Ethereum Classic – Crypto-anarchists view

*The terms of The DAO Creation are set forth in the smart contract code existing on the Ethereum blockchain at 0xbb9bc244d798123fde783fcc1c72d3bb8c189413. **Nothing in this explanation of terms or in any other document or communication may modify or add any additional obligations or guarantees beyond those set forth in The DAO’s code.** Any and all explanatory terms or descriptions are merely offered for educational purposes and do not supercede or modify the express terms of The DAO’s code set forth on the blockchain; to the extent you believe there to be any conflict or discrepancy between the descriptions offered here and the functionality of The DAO’s code at 0xbb9bc244d798123fde783fcc1c72d3bb8c189413, The DAO’s code controls and sets forth all terms of The DAO Creation.*

WK 03-01

23



- The DAO (Decentralised Autonomous Organisation) attack is a classic example of reliance only on code and governance issues around it.
- The DAO was launched in April 2016 as a decentralised venture capital fund.
 - The founders of The DAO established a Swiss-based company, DAO. The link was in Switzerland because Swiss law allowed it to "take money from an unknown source as long as you know where it's going.
- Members contributed ETH and received DAO tokens in return. It raised \$150 million in ETH in a month.
- But a “bug” in the code led to a slow leak of > \$50M.
- This led to developers “forking” the Ethereum blockchain to recover funds, i.e., the state associated with the DAO contract was reset by chaining the Ethereum protocol and returning those ETH to the contract. Miners that didn’t like the fork continued to use the same ledger state splitting the Ethereum into 2 networks, Ethereum and Ethereum Classic.
- So, in 2016, we saw these 2 world views (governing code with the law vs power of code taking over from the law) dramatically confront each other.
- On the right, we see this legalistic disclaimer in The DAO’s whitepaper. It follows the crypto-anarchist worldview, saying that the running code is all that matters.
 - If the specification for the program is the program itself, can a program even have a bug?
- The response of the Ethereum community was fascinating. There was a lot of debate, but ultimately developers changed the code of Ethereum itself to remediate the effect of this bug.
 - They changed the code to reflect their values and the values of the Ethereum community. This change to the code created what is called a “hard fork” of Ethereum.
 - There are now two alternative blockchains, Ethereum today has the fix, and Ethereum Classic doesn’t.
 - Ethereum is worth almost a hundred times as much as Ethereum Classic, which indicates that the community largely agreed with the fix.
- Lessig was right in that we can choose values for our code, even if this means changing the code.
- But Ethereum Classic continues, and so the crypto-anarchists are also choosing those values for their community.

Smart Legal Contracts



Smart contracts can operationalise legal contracts

- “help to administer/automate the performance of”

Can some smart contracts also be legal contracts?

Can some smart contracts be clauses in a legal contract?

- Is the program (part of) the contract itself?
 - Specification? Source code? Bytecode? (Not the running machine!)
 - Which interpretation matters? The courts, business parties, programmer, or the machine?
- Does it matter if parties or the judge don’t understand the language?

- Smart contracts can operationalise legal contracts. They can help to administer or automate the performance of a certain action in a legal contract.
 - E.g., following Nick Szabo’s example of a leased car, if the owner misses 3 consecutive payments, the owner’s ability to open the door can be locked while giving access to the creditor.
- An important question is “can some smart contracts also be legal contracts”?
- It is more likely that some smart contracts be clauses in a legal contract.
- If so, the important question is are we considering the program written in some language to be that clause or some specification of what that program should be the clause?
 - This gets further complicated when the source code is compiled with different compiler options. E.g., there are slight differences in how an Ethereum smart contract runs when they are compiled with targeting different versions of EVM. Therefore, one may consider the bytecode than the source code.
 - Even the same bytecode may behave differently on different systems, e.g., different versions of EVM behave slightly differently. So, while people usually consider only the code, even the machine it runs can have an impact.
- Even the same code (source code or bytecode) will be interpreted differently by different parties and affect how the specification translates into a program.
 - How a smart contract is interpreted by a machine would be very different from a programmer’s. The machine may interpret it differently from the programmer, and the ultimate results depend on how the machine interprets it.
- Does it matter if the parties or the judge don’t understand the language? Probably not, as you would bring in a domain expert during the dispute resolution process.

Agenda

- Smart Contracts
- Smart contracts and legal contracts
- Tokens
- Oracles

Tokens

Assets you can hold, not to be copied, to

- Use once or Reuse?, Transfer?, or Sell?

Token issuer says it represents

- Authorisation?
- A license to use or do something?
- Ownership of property?

All of these are some kinds of rights



- Tokens are assets you can hold. They are bearer instruments as one who holds them owns them.
- They are expected to be not copied by the bearer or anyone else and use various physical or logical mechanisms to make them harder to copy, e.g., seal, embossed pattern, secure paper, etc.
- Token issuer may specify that a token represents one or more things like:
 - Authorisation to perform some action, e.g., admit 1 person to a movie.
 - A license to use or do something, e.g., a license to drive a car.
 - ownership of a property, e.g., deed and poker chips
- All of these are some kind of a right that the token issuer grants you.

Tokens – Cont.

“Any digital representation of an interest, which may be of value, a right to receive a benefit or perform specified functions, or may not have a specified purpose or use” – European Securities and Markets Authority

Tokenisation

- Method that converts rights to an asset into a digital token, usually on a DLT
- Benefits
 - Automation, Disintermediation, Transparency, Improved liquidity, Efficient clearance & settlement

- There are multiple definitions of a token and tokens in digital form. Among them, the European Securities and Markets Authority says a token is:
 - Any digital representation of an interest, which may be of value, a right to receive a benefit or perform specified functions or may not have a specified purpose or use.
 - This definition also captures the idea that a token may be just a token and may not represent any useful purpose, or its purpose or use will be specified in the future. This is the case for some of the tokens issued on public blockchains.
- Tokenisation is the process that converts rights to an asset into a digital token, usually on a DLT. Sometimes legal recognition is required for the token right to be effective.
- Tokenisation drives the financialisation of an asset by creating a “bridge” between the asset and the token.
 - The essence of these processes is that the economically and legally most important features of an asset are recorded and that the written record is treated as an object that can be bought and sold on the secondary market.
- Benefits of tokenisation include :
 - Automation – Management of the token can be automated through smart contracts (of course someone still needs to send a transaction to trigger relevant smart contract functions).
 - Disintermediation – Does not require or reduce the need for 3rd parties, e.g., an Escrow contract can manage the token transfer.
 - Transparency – All mining, transfer, and burning operations on tokens are visible as per the access control settings of the blockchain platform.
 - Improved liquidity – Make it more easily tradable. E.g., tokens are good for managing fractional ownership, so while it may be difficult to sell an entire property setting a small fraction of that is less difficult.
 - *Liquidity* refers to the ease with which an asset can be converted into cash, e.g., gold and stocks are more liquid than a land title.
 - Efficient clearance and settlement – Like most digital things, when tokens are digitalised, they are easy to manage. Hence, we can more efficiently clear and settle token-based transactions.
 - In financial markets, *clearing* is the process of recording the transaction and arranging for the transfer of money and securities/assets. *Settlement* is the actual exchange of money, or some other value, for the securities.

Cryptocurrency vs Token

Cryptocurrencies & tokens are both digital assets, on a shared ledger

Cryptocurrency	Tokens
Is baked into blockchain platform by developers, aka native currency	User-programmable in smart contracts
Accounting & validation rules are hard-coded in the platform's base layer	Accounting & validation rules are defined in token smart contract(s)
Not centrally issued, e.g., Bitcoin & Ether	Centrally issued by someone, e.g., Tether & Mintable NFTs
Don't represent other rights/assets	Represent some kind of rights
Usually only on public blockchains	Usually on public & private blockchains

WK 03-01

28



- Cryptocurrencies and tokens both are digital assets on a shared ledger. However, they have several notable differences.
- Cryptocurrency is baked into the base layer of the blockchain by its developers while tokens (e.g., stablecoins) are user-programmable and defined in smart contracts.
 - However, some of the ERC-20 tokens are also called coins or cryptocurrencies when they are used as a medium of exchange, e.g., Binance token (BNB) and USD Coin (USDC), Dai (DAI).
 - Because cryptocurrency is part of the blockchain platform it's also called "native currency".
- Cryptocurrency accounting and validation rules are hard-coded in the platform's base layer. Whereas in tokens, accounting and validation rules are defined in the respective smart contracts.
- Cryptocurrencies aren't issued by any entity and users must effectively rely on the software protocol that controls the system, e.g., Bitcoin & Ethereum. Tokens such as Tether and NFTs from Mintable.
 - So, on your own, you can issue a token but not base cryptocurrency.
- While cryptocurrencies don't represent any kind of right or other assets, tokens represent some kind of rights.
- Cryptocurrencies are usually issued only on public blockchains while tokens are issued on both public and private blockchains.
 - There are efforts to implement tokens on centralised systems, e.g., CBDCs.

Token Categories

Fungible tokens

Identical & interchangeable

- E.g., \$2 coin, \$10 note

Can be split

Main concern: How many?



Binance Coin
(BNB)



Tether USD
(USDT)



Perth Mint Gold Token
(PMGT)

Non-fungible tokens (NFTs)

Unique

- E.g., houses, art, patents

Can't be split

Main concern: Which ones?



CryptoKitties Dragon
\$1.3M



Bored Ape #2087
\$2.3M



Beeple's opus
\$69M

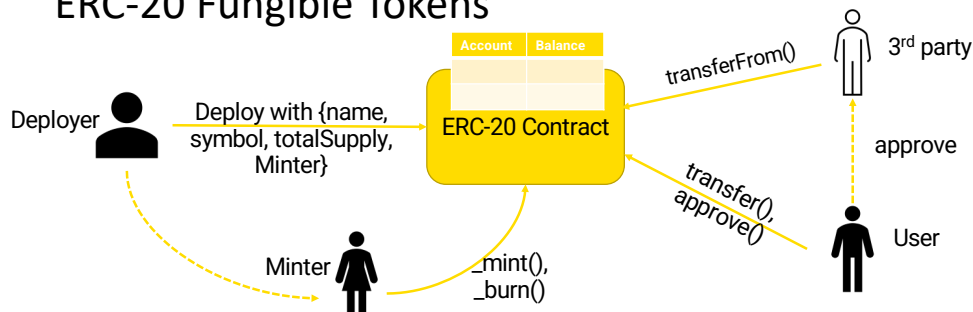
WK 03-01

29



- Tokens represent some sort of ownership, e.g., in the good old days, money notes represented the Gold maintained by Central/Reserve banks.
 - Tokens can be broadly classified as fungible and non-fungible.
 - Fungible tokens
 - One token is identical to another hence they can be interchanged, e.g., think of 2 coins or notes. They represent the same value and can be used in place of the other.
 - These tokens can be usually split into smaller values. E.g., we can maintain fractional ownership by defining the decimal points.
 - The main concern with such tokens is how many one owns.
 - Lots of cryptocurrencies are offered on top of Ethereum as ERC-20 tokens.
 - Non-fungible tokens (NFTs)
 - NFTs record the ownership of a unique tangible or intangible object; hence, each token is unique, e.g., we can't interchange a house and its land with another.
 - Usually, these tokens can't be split. E.g., we don't maintain ownership of land separate from a permanent structure on that land.
 - However, we may record fractional ownership against the same token without splitting. E.g., a couple can own a property.
- With NFTs, our concern is which token as some tokens are more valuable than others due to the rights, they represent is different. For e.g., while a few Cryptokitties (<https://www.cryptokitties.co/>) are sold for \$100K+ most kitties worth only a few \$10s to \$100s. Kitties are non-fungible, individual, and their appearance depends on individual features.

ERC-20 Fungible Tokens



```
transfer(address _to, uint256 _value) public returns (bool success);
transferFrom(address _from, address _to, uint256 _value) public returns
(bool success);
approve(address _spender, uint256 _value) public returns (bool success);
_mint(address account, uint256 amount);
_burn(address account, uint256 amount);
```

See <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC20/ERC20.sol>

- Here is an example of rights/powers/capabilities embedded in a token (actually in the smart contracts that manage the token).
- ERC-20 (Ethereum Request for Comments 20 arising from Ethereum Improvement Proposal (EIP) 20) is the most popular fungible token standard.
- Someone deploys the ERC-20 token contract while setting a few parameters like token name, symbol, total supply, and minter. Minter can be skipped if the total supply is created during deployment.
- The contract keeps track of token account balances.
- It exposes functions where the token owner can transfer tokens (*transfer* function) and approve a 3rd party to spend tokens on behalf of itself up to a given limit (*transferFrom* function).
- Then a 3rd party can transfer from an approved account up to the approved limit.
- Then there are private functions for the token contract owner to mint and burn tokens.

ERC-721 NFTs

```
transferFrom(address _from, address _to, uint256 _tokenId) external payable;
safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes data)
external payable;
approve(address _approved, uint256 _tokenId) external payable;
setApprovalForAll(address _operator, bool _approved) external;
_mint(address to, uint256 tokenId);
_burn(uint256 tokenId);
```



Token ID	Address

Tracks token ownership, but not what a token represents.
Token DApp needs to track what token represents, e.g.,
`_setTokenURI(uint256 tokenId, string _tokenURI);`

See <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC721/ERC721.sol>

- ERC-721 non-fungible tokens (most popular NFT standard) specification is very similar to ERC-20 with a few additional functions.
- Like ERC-20 it specifies the token owner can transfer tokens (*transfer* function) and approve a 3rd party to spend a token on its behalf (*transferFrom*). It may also approve all tokens or remove any existing approval (*approve* function).
- Then a 3rd party can transfer the token from an approved account.
- *safeTransferFrom* checks that contract recipients are aware of the ERC721 protocol to prevent tokens from being forever locked/lost.
- A key difference between this and ERC-20 contracts is how it keeps track of the ownership of the asset. In ERC-20, as each token is identical, we track the token balance of an account. Whereas in ERC-721, each token is unique. Hence, we keep track of the token and its owner.
- However, the contract does track what a token represents. Such information must be tracked by the token application or another contract that is expected to extend from the ERC-721 token contract. E.g., we can use a function like `_setTokenURI(uint256 tokenId, string _tokenURI);` to keep track of the URI/URL of the underlying asset
 - A Uniform Resource Identifier (URI) is a unique sequence of characters that identifies a logical or physical resource used by web technologies.
- However, this may still not indicate the rights and obligations of token.

Agenda

- Smart Contracts
- Smart contracts and legal contracts
- Tokens
- Oracles

- We start with a discussion on smart contracts and their relationship (or lack of relationship) with legal contracts.
- Next, we'll have a very high-level discussion about tokens.
- Finally, we'll discuss oracles that enable a blockchain/smart contract to interact with off-chain components of a software system.

Oracles

Value of blockchain is the difference it can make to our lives off-chain

On-chain actions may depend on what happens in the real world

- E.g., agriculture insurance payouts via a smart contract need weather data

Off-chain actions may depend on what happens on-chain

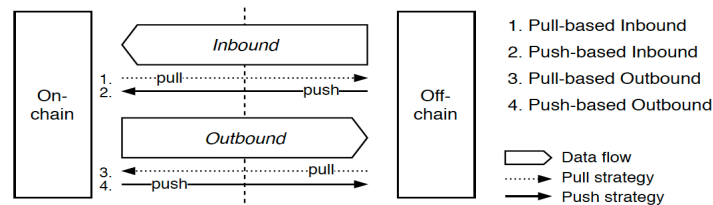
- E.g., transfer ownership of physical asset given transfer of its digital counterpart

Oracles enable data flow between blockchain & real world

- Usually run by 3rd parties
- Has off-chain or on-chain & off-chain components
- Centralised point of failure, & may introduce security & trust concerns

- We already discussed that the value of blockchain is the difference it can make to our lives off-chain.
- On-chain actions may depend on what happens in the real world.
 - E.g., agriculture insurance payouts via a smart contract need weather data to determine whether there was a significant hail event to pay insurance automatically.
- Whereas off-chain actions may depend on what happens on-chain.
 - E.g., transfer ownership of physical assets given transfer of its digital counterpart. This could even be that by noting an on-chain (crypto) payment, one may transfer off-chain asset ownership.
- *Oracles* are components that enable data flow between blockchain and the real world.
 - They are very important components of a blockchain-based application and are usually run by a third party.
 - They may appear as hardware or software components, as well as human oracles.
 - Oracles may be implemented purely off-chain components or as a combination of on-chain and off-chain components
- However, they are widely criticised due to reasons such as:
 - Potential for centralised/single point of failure.
 - The potential for introducing security and trust concerns.
- Oracle code can be quite long and complicated as it involve on-chain and off-chain code. We'll go through a simple example during one of the lab sessions.

Data Flow in Oracles

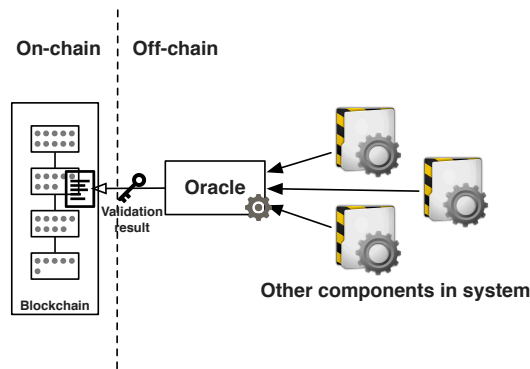


	Pull	Push
Inbound	On-chain component requests off-chain state from an off-chain component	Off-chain component sends off-chain state to on-chain component
Outbound	Off-chain component retrieves on-chain state from an on-chain component	On-chain component sends off-chain state to an off-chain component

Source: R. Mülhberger et al., "Foundational oracle patterns: Connecting blockchain to the off-chain world", In Int. Conf. on Business Process Management, Sep. 2020.

- Data flowing into a blockchain from the real world is called *inbound data*.
- Data flowing out of a blockchain into the real world is called *outbound data*.
- Each of these data flows can be facilitated by push or pull techniques. While these will be somewhat explained in the next couple of slides, the following is a short explanation:
- In pull-based inbound, an on-chain component requests an off-chain state/data from an off-chain component.
 - This is usually implemented by a smart contract emitting an event that the off-chain component listens to. The off-chain component then submits the requested data.
 - From the point of view of the on-chain components, this is like pulling data from the real world.
 - E.g., a smart contract may want to know whether a physical asset was delivered to pay on-chain.
- In push-based inbound, the off-chain component sends the off-chain state to the on-chain component.
 - This is implemented by the off-chain component pushing data to a smart contract as and when off-chain data get updated.
 - From the point of view of the on-chain components, this is like an off-chain component pushing data from the real world.
 - E.g., a price oracle may automatically report the ETH:USD rate of a smart contract when the price changes by 1%.
- In pull-based outbound, an off-chain component retrieves an on-chain state from an on-chain component
 - This is implemented by an off-chain component pulling data from a smart contract or blockchain when it needs some on-chain data.
 - From the point of view of the off-chain components, this is like an off-chain component pulling data from the blockchain.
 - E.g., a shipping company may want to know if payment was done on-chain before handing over a physical asset to the buyer. This can be done by calling a *view* function on the smart contract.
- In push-based outbound, an on-chain component sends an off-chain state to an off-chain component
 - This is usually implemented by a smart contract emitting an event that the off-chain component listens to.
 - From the point of view of the off-chain components, this is like the on-chain component pushing data to the real world.
 - E.g., when the payment is made on-chain a smart contract may emit a "paid" event.
- These 4 patterns have varying properties which determine their suitability for a given application, e.g.,
 - Pull-based inbound oracles are slow as an event needs to be emitted before sending data on-chain using a transaction. This transaction may take a long time to get included in a block.
 - Whereas push-based oracles reduce the latency as they proactively send data to a smart contract. However, the transaction cost of frequent pushing can be high and increases the number of transactions to be included.

(Centralized) Oracle



Introduces external state into blockchain using an oracle

Oracle injects state to blockchain in a TX signed with its private key

Validation of TX is based on oracle's identity/address

Drawbacks

- Oracle must be trusted (usually a 3rd party)
- No way to validate external state injected by oracle
- Single point of failure

- A centralised oracle (aka oracle) is a software pattern that allows data flow from the real world to the blockchain.
- **Context:** From the software architecture perspective, blockchain can be viewed as a component or connector within a large software system.
- In this case, the blockchain is used as a distributed database for more general purposes other than financial services.
- The applications built on the blockchain might need to interact with other external systems.
- Thus, the validation of transactions on the blockchain might depend on the states of external systems.
- **Problem:** The execution environment of a blockchain is self-contained.
- It can only access information present in the data and transactions on the blockchain.
- Smart contracts running on the blockchain are pure functions by design.
- The state of the external system is not directly accessible to smart contracts.
- Yet, function calls in smart contracts sometimes need to access the state of the external world.

Forces

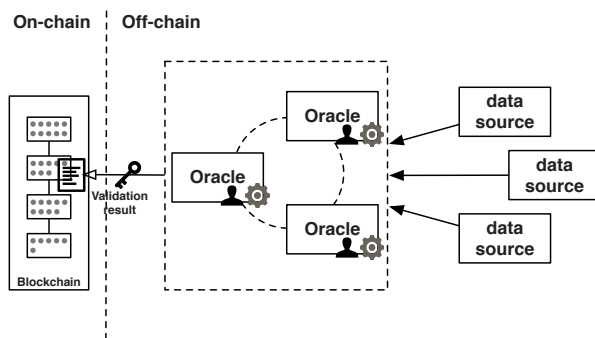
- Closed environment. blockchain is a secure, self-contained environment, which is isolated from external systems.
- Smart contracts on blockchain cannot read the states of the external systems.
- Connectivity. In addition to the data found on the blockchain, general-purpose applications might require information from external systems.
- For example, a parcel tracking application needs context information like geo-location information, and a gambling application might need the result of a football game from a Web API.
- Long-term availability and validity. While transactions on the blockchain are immutable,
- the external state used to validate a transaction may change or even disappear after the transactions were originally appended to the blockchain.
- **Solution:** To connect the closed execution environment of blockchain with the external world, oracle is introduced to assist in evaluating conditions that cannot be expressed in a smart contract running within the blockchain environment.
- An oracle is a trusted third party that provides smart contracts with information about the external world.
- When validation of a transaction depends on the external state, the oracle is requested to check the external state and inject the result to the blockchain in a transaction signed using its key pair.
- The validators (miners) take the result provided by the oracle into account when validating the transaction.
- Through using an oracle, the validation of transactions is based on the authentication of the oracle, rather than the external state.
- Connectivity. The closed execution environment of blockchain is connected with the external world through the oracle.
- The applications based on the blockchain can access external states through the oracle and use the external states to validate transactions.
- Trust. Using oracle introduces a trusted third party into the system.
- The oracle selected to verify the external state needs to be trusted by all the participants involved in relevant transactions.
- Validity. The external states injected into the transactions can not be fully validated by miners.
- Thus, when miners validate the transaction including the external state, they rely on the oracle to check the validity of the information from the external world.
- Long-term availability and validity. It could happen that while transactions are immutable,

- the external state used to validate them may change after the transactions were originally appended to the blockchain.

Known use:

- Oracle in Bitcoin is an instance of this pattern
- Oracle is a server outside the Bitcoin network, which can evaluate user-defined expressions based on the external state.
- Oraclize is an oracle service provider, which utilises trusted hardware to directly fetch information from an external trusted execution environment (TEE).
- Oraclize introduces three different proofs for fetching data from external data sources, namely, TLS-Notary, Ledger proof and Android proof.
- Corda has a centralized oracle mechanism embedded in its platform. The oracle mechanism uses Intel Software Guard Extensions (SGX) for hardware attestation to prevent unauthorised access outside of the SGX environment.

Decentralized Oracle



Introduces external state into blockchain using multiple oracles

- Each TX signed by respective private key
- Consensus using voting or m-out-of-n multi-signatures
- Ideally, data should come from independent data sources

Enhance trust, reliability, & availability

High cost & slow

Oracles may still collude

- In this design, instead of a single oracle, we rely on a group of oracles ideally managed by independent 3rd parties.
- **Context:** Like the centralized oracle, blockchain is used as a distributed database for more general purposes other than financial services.
- The applications built on the blockchain might need to interact with other external systems.
- the validation of transactions on blockchain relies on oracle to inject the states of external systems into the blockchain.
- **Problem:** The execution environment of a blockchain is self-contained. It can only access information present in the data and transactions on the blockchain.
- smart contracts running on the blockchain are pure functions by design. The state of external systems is not directly accessible to smart contracts.
- Yet, function calls in smart contracts sometimes need to access the state of the external world.
- However, a centralized oracle introduces a single trusted third party into the system, which might become a single point of failure.
- **Forces:** Reliability. A centralized oracle becomes a single point of failure from an architectural perspective.
- If the status injected into the blockchain is faulty, then the whole system is unreliable.
- Variety of data sources. The data source(s) could be a static web page, a physical sensor, a component of a system or even input from a human.
- Decentralized oracles request data from various sources.
- Despite having multiple data sources, some information such as the owner of a property from different resources might ultimately come from a single authorised source.

Solution: To improve the trustworthiness of the oracle, a decentralized oracle mechanism is introduced, which is based on multiple servers and multiple data sources. K-out-of-M threshold signature can be used by multiple oracles to reach a consensus on the status to be injected into the blockchain.

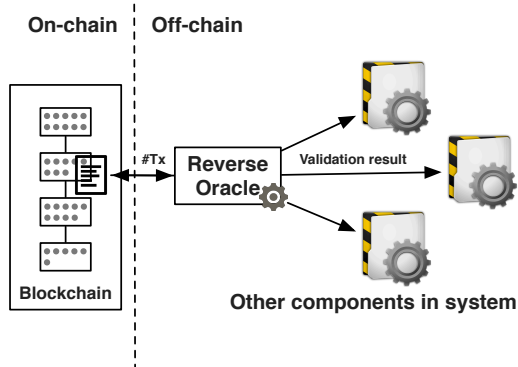
For example, a 3-out-of-5 signature scheme requires at least three or more oracles out of five oracles to sign on the same value for the value to be injected into the blockchain.

- **Reliability.** By having multiple oracles injecting values into the blockchain, the risk is reduced from a single point of failure.
- Acquiring data by multiple oracles also improves the reliability of the final accepted value.
- The oracles are allowed to fetch the requested data from multiple external data sources, improving the likelihood of getting

accurate external data.

- Trust. Although using a decentralized oracle avoids the single point of failure, it still introduces trusted third parties into the system.
 - All the oracles that verify the external state needs to be trusted by all the participants involved in relevant transactions.
 - Cost. Cost increases with the number of oracles being used.
 - Time. It might take a longer time for multiple oracles to get the required information and reach a consensus for the final result.
-
- Orisi on Bitcoin maintains a set of independent oracles. Orisi allows the participants involved in a transaction to select a set of oracles and define the value of M before initiating a conditional transaction.
 - Gnosis is a decentralized prediction market that allows users to choose any oracle they trust, such as another user or a web service, e.g., for weather forecasts. A human oracle is also called an arbitrator, who is trusted by the interacting participants to resolve disputes or check the external state.
 - Augur is another prediction market that leverages the capability of human oracles to do predictions and resolve disputes.

Reverse Oracle (Pull-based Oracle)



Off-chain component rely on smart contract to check conditions & provide requested data

Performs the reverse process of an oracle

Push or pull data

- Call smart contract functions – Pull
- Subscribe to events – Push

Additional logic is needed to handle probabilistic finality

- In this design, the data flow is from blockchain into the real world.
- **Context:** In a software system, where blockchain is one of the components, the off-chain components might need to use the data stored on the blockchain, and the smart contracts running on the blockchain to check certain conditions.
- Problem: Some domains use very large and mature (or even legacy) systems, which comply with existing standards.
- For such domains, a non-intrusive approach is desired to leverage the existing complex systems with blockchain without changing the core of the existing systems. How to integrate the blockchain within existing systems?
- Connectivity. Integrating blockchain into an existing system to leverage the unique properties of the blockchain.
- Simplicity. Introduce minimal changes to the existing system.
- **Solution:** The unique ID of the blockchains or blocks on the blockchain is a piece of data that can be easily integrated into the existing systems.
- Validation of the data can be implemented by smart contracts running on the blockchain.
- An off-chain component is required to query the blockchain by using the ID of the data by calling respective functions on the smart contract.
- Alternatively, smart contracts can emit events when a certain condition is achieved, which the off-chain components can subscribe to.
- Connectivity. The blockchain is integrated into an existing system by adding the ID of the transaction as a piece of data into the system and using smart contracts to do data validation.
- Non-intrusive. It is not always possible to use a blockchain in a non-intrusive way depending on the extensibility of the existing system. In particular, the probabilistic commit of blockchains using Nakamoto consensus may be inconsistent with normal transaction semantics in enterprise systems. Additional logic in the reverse oracle component may be required to cover these differences.
- Finality. When transactions are not final as soon as they are included in a block, we need to wait until the desired number of confirmation blocks are built before assuming on-chain data/event is persistent.
- Identity provides a solution to enrich the payments in banking systems with documents and attributes through using blockchain.
- Identity invents the concept of an identity token stored on a blockchain.
- Every payment is associated with an identity token, which is used to exchange enriched information about a payment.
- The identity token is exchanged between the banks by being embedded into the SWIFT protocol.
- Slock.it is aimed to build autonomous objects and a universal sharing network by using blockchain and IoT devices.
- where the devices can sell or rent themselves and pay for services provided by others. In terms of renting a device, the availability information is stored on the blockchain.
- validity checking can be done on blockchain.

Question

Mark True or False for each the following statements about smart contracts and oracles.

	True	False
Not all smart contracts belong to a DApps	✓	
An asset can only be tokenised as an NFT		✓
Smart contracts can operationalise legal contracts	✓	
A centralised oracle enhances trust, availability, and cost		✓

- A DApp is an application where main functionality is implemented through smart contracts. However, this does not mean that all smart contracts must be part of a DApp.
- Depending on the asset type it can either be a fungible or non-fungible token
- smart contracts can be used to execute part of a legal contract, e.g., automatically releasing funds when payment conditions are satisfied.
- A centralised oracle can lead into a single point of failure, and we can't trust a single party. Whereas a decentralised oracle with multiple parties has better availability and trust. Cost discussion is reversed where centralised is less costly due to one transaction per update, whereas decentralised oracles will have multiple transactions per update.